

Day 3 - LinkedList Cycle

ITSRUNTYM

Problem Understanding - Cycle Detection in LinkedList

We're given the head of a linked list, and we need to:

1. **Detect if there is a cycle.**
2. If yes, **find the node where the cycle begins.**

Example

```
1 → 2 → 3 → 4 → 5
      ↑      ↓
      8 ← 7 ← 6
```

Here:

- Cycle exists (3 → 4 → 5 → 6 → 7 → 8 → 3).
- Start of cycle = 3.

2. Approaches

Approach 1 — Using Hashing (Extra Space)

Idea

- Use a HashSet to store visited nodes.
- While traversing, if we encounter a node already in the set, we've found the cycle's start.
- If we reach null, no cycle exists.

Efficiency

- Time: **O(n)** (visit each node once).
- Space: **O(n)** (store visited nodes).

Approach 2 — Floyd's Cycle Detection Algorithm (Tortoise & Hare)

Idea

- **Phase 1 (Detect Cycle):**
Use two pointers — slow moves 1 step, fast moves 2 steps.
If they meet → cycle exists.
If fast reaches null → no cycle.
- **Phase 2 (Find Start of Cycle):**
When a meeting point is found:
 - Place one pointer at head and keep the other at meeting point.

- Move both one step at a time.
- Where they meet again → start of the cycle.

Why this works (Math Explanation)

Let:

- $L1$ = distance from head to start of cycle.
- $L2$ = distance from start of cycle to meeting point inside cycle.
- C = cycle length.

When slow and fast meet:

$$\text{Distance(slow)} = L1 + L2$$

$$\text{Distance(fast)} = L1 + L2 + k \cdot C \quad (k \text{ is number of full cycles by fast})$$

Since fast is twice as fast:

$$2(L1 + L2) = L1 + L2 + k \cdot C$$

$$L1 = k \cdot C - L2$$

This means if one pointer starts from head and another from meeting point, moving at same speed, they meet at cycle start.

Efficiency

- Time: **$O(n)$** (single traversal).
- Space: **$O(1)$** .

3. Code Implementation

Approach 1 — Hashing

```
import java.util.HashSet;

class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}

public class DetectCycleHashing {
    public ListNode detectCycle(ListNode head) {
        HashSet<ListNode> visited = new HashSet<>();
        ListNode curr = head;
        while (curr != null) {
            if (visited.contains(curr)) {
                return curr; // start of cycle
            }
            visited.add(curr);
            curr = curr.next;
        }
    }
}
```

```

        return null; // no cycle
    }
}

```

Approach 2 — Floyd's Algorithm (Optimized)

```

public class DetectCycleFloyd {
    public ListNode detectCycle(ListNode head) {
        if (head == null || head.next == null) return null;
        ListNode slow = head, fast = head;

        // Phase 1: Detect cycle
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) { // Cycle detected

                // Phase 2: Find start of cycle
                ListNode start = head;
                while (start != slow) {
                    start = start.next;
                    slow = slow.next;
                }
                return start; // Start of cycle
            }
        }
        return null; // No cycle
    }
}

```

4. Example Walkthrough

Let's test on:

```

1 → 2 → 3 → 4 → 5
    ↑      ↓
    8 ← 7 ← 6

```

- Slow: moves 1 step.
- Fast: moves 2 steps.
- They meet at node 6.
- One pointer starts from head (node 1), other from meeting point (6).
- They both meet at node 3 → cycle start.

5. Comparison

Approach	Time	Space	Easy to Code	Best Use
----------	------	-------	--------------	----------

Hashing	$O(n)$	$O(n)$	Easy	When memory isn't a concern
Floyd's	$O(n)$	$O(1)$	Slightly tricky	Always best for large lists